Geometric Language Model: A Research Dossier

Working Label: GLM-3 (Geometric Language Model, Strict 3D-State Prototype)

Technical Working Research Document

October 27, 2026

Abstract

This document records the proposed architecture and technical analysis for a language model with 3D geometric representations. The central proposal has two main commitments: (1) Representation: each token's contextual state is fundamentally a point in ordinary three-dimensional Euclidean space $\mathbb{R}^3$, and (2) Relations: concepts influence one another through geometric relations such as distance, transformed distance, and weighted aggregation. We present mathematical specifications, formal algorithmic procedures, detailed architectural components, worked examples, a complete dedicated baseline PyTorch prototype, an experimental protocol, and an extensive prior research analysis to evaluate whether strict 3D contextual states can represent natural language dependencies.

1 Executive Summary and Research Question

The proposal is to investigate whether a language model can represent each token state as a point in $\mathbb{R}^3$ and model token-to-token influence using geometric relations, rather than relying on the high-dimensional dot-product attention mechanism used in standard Transformers.

The most direct first prototype is an autoregressive model. It embeds tokens and positions into $\mathbb{R}^3$, computes a causal pairwise geometric score, normalizes scores over the visible prefix, aggregates messages, updates each token's 3D state, and decodes the state into vocabulary logits.

Conceptual Data Flow:

Token IDs → 3D Token + Position → Geometric Relation Scores

→ Causal Softmax → Weighted Messages → Residual + Update

→ Vocabulary Decoder → Next-Token Logits

Table 1: Initial Feasibility and Claims Assessment

Level / Claim	Assessment & Evidence Needed
Mathematical feasibility	**Yes.** Operations are well-defined/differentiable. Shape checks, equations, stable gradients.
Task usefulness	**Plausible.** A small model may learn toy grammar, local patterns, and limited relational tasks.
Architectural advantage	**Unproven.** Strict 3D states create a severe bottleneck; pairwise attention still costs $\mathcal{O}(L^2)$.
Prior novelty	**Not established.** Geometric embeddings and relation transforms exist across literature.
Next step	Build prototype, pass tests, compare against carefully matched baselines.

1.1 Core Research Question

Can a model whose contextual state is strictly three-dimensional learn meaningful language dependencies, and can geometric relation functions make that representation more effective than other mechanisms under the same bottleneck?

The strongest defensible framing today is: a testable geometric language-model hypothesis, not yet a proven replacement architecture.

2 Background: How a Transformer Works

2.1 From Text to Tokens

A tokenizer converts text into token IDs (words, word pieces, punctuation, or bytes). For example, "Cats chase small birds." becomes `[Cats, chase, small, birds, .]`.

2.2 Embeddings and Positional Information

Token IDs map to learned vectors wider than 3D in standard Transformers. Positional information is injected so the model distinguishes sequence order (e.g., "dog bites person" vs "person bites dog").

2.3 Self-Attention

For sequence matrix X, standard Transformers compute Query (Q), Key (K), and Value (V) projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V \tag{1}$$

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \text{causal_mask}\right), \quad Y = AV \tag{2}$$

2.4 What the Geometric Proposal Changes

The proposal replaces high-dimensional token states and dot-product compatibility with 3D states and a geometric score (e.g., negative squared distance). However, computing pairwise scores, softmax weights, and weighted sums keeps it in the broad attention family.

3 Proposed Architecture and Design Commitments

3.1 Working Name: GLM-3

GLM-3 refers to a strict 3D-state geometric language model where each token's state has exactly three scalar components throughout all geometric blocks.

3.2 Input Representation

Let a token sequence be $t_1, \ldots, t_L$. Each token receives a learned embedding $e(t_i) \in \mathbb{R}^3$ and position embedding $p_i \in \mathbb{R}^3$:

$$x_i^{(0)} = e(t_i) + p_i \in \mathbb{R}^3 \tag{3}$$

3.3 Geometric Relation Score

For receiver state x_i and source state x_j, with learnable 3×3 transform R:

$$s_{ij} = -\frac{\|x_i - Rx_j\|^2}{\tau} \quad (\tau > 0) \tag{4}$$

3.4 Causal Normalization

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{k \le i} \exp(s_{ik})} \quad (j \le i); \quad \alpha_{ij} = 0 \quad (j > i) \tag{5}$$

3.5 Message Aggregation and State Update

With learnable 3×3 message matrix A and nonlinear update $\varphi : \mathbb{R}^3 \to \mathbb{R}^3$:

$$m_i = \sum_{j \le i} \alpha_{ij} A x_j \in \mathbb{R}^3 \tag{6}$$

$$x_i' = \text{LayerNorm}(x_i + \varphi(m_i)) \tag{7}$$

3.6 Output Decoder

The prototype maps 3D state to V vocabulary logits using a non-linear MLP ($3 \to 64 \to V$).

Table 2: Architectural Variants and Consequences

Variant	Scientific Consequence
Strict point	Most faithful to hypothesis; strongest bottleneck.
Point + orientation	Richer state; no longer only three scalar degrees.
Multiple 3D primitives	Increases capacity and complexity per token.
3D + hidden features	Tests a different claim from strict 3D.

4 Detailed Component Breakdown

To thoroughly understand the inner mechanics of GLM-3, each subsystem is analyzed in isolation:

4.1 1. Geometric Token & Position Embeddings

In traditional language models, embedding tables project discrete indices into high-dimensional space $\mathbb{R}^d$ ($d \in [512, 4096]$). In GLM-3, $e(t) \in \mathbb{R}^3$. Position vectors $p_i \in \mathbb{R}^3$ are added component-wise. This initializes sequence elements as points in a bounded 3D bounding volume.

4.2 2. Linear Metric Transformation (R)

Rather than raw Euclidean distance $\|x_i - x_j\|^2$, GLM-3 maps source tokens through a 3×3 transformation matrix R. R acts as an affine coordinate transformation allowing rotation, scaling, and shear before measuring Euclidean proximity.

4.3 3. Temperature Scaling (τ)

Because squared Euclidean distances scale quadratically, temperature τ controls the softness of attention distributions. We parameterize τ via $\tau = \text{softplus}(\hat{\tau}) + \epsilon$ to enforce positivity and prevent infinite concentration or uniform collapse.

4.4 4. Causal Masking Mechanism

Causal masking guarantees autoregressive consistency. Future token positions ($j > i$) are masked with $-\infty$ prior to softmax computation, forcing the upper-triangular affinity matrix to zero weight.

4.5 5. Message Matrix Projection (A) & Update MLP (φ)

The linear transform $A \in \mathbb{R}^{3\times3}$ extracts contextual messages from source points. Aggregated messages m_i pass through a two-layer MLP with nonlinear activation φ, which is added back to x_i via residual connection and normalized using LayerNorm.

4.6 6. Vocabulary Projection Subsystem

The final 3D coordinates $x_i^{(N)}$ pass into a 2-layer MLP decoder ($3 \to 64 \to V$). Because the $3D$ coordinate space is constrained, the non-linear decoder plays a crucial role in forming decision boundaries over the discrete vocabulary V.

5 Mathematical Specification

5.1 Shapes and Dimensions

- B: Batch size, L: Sequence length, V: Vocabulary size
- $E \in \mathbb{R}^{V\times3}$: Token embeddings, $P \in \mathbb{R}^{L_{\max}\times3}$: Position embeddings
- $X \in \mathbb{R}^{B\times L\times3}$: Token states, $R, A \in \mathbb{R}^{3\times3}$: Linear transforms
- $S, \alpha \in \mathbb{R}^{B\times L\times L}$: Pairwise scores and attention weights
- $M \in \mathbb{R}^{B\times L\times3}$: Aggregated messages, $Z \in \mathbb{R}^{B\times L\times V}$: Vocabulary logits

5.2 Pairwise Distance Calculation

If $x_i = (a_1, a_2, a_3)$ and $Rx_j = (b_1, b_2, b_3)$:

$$\|x_i - Rx_j\|^2 = (a_1 - b_1)^2 + (a_2 - b_2)^2 + (a_3 - b_3)^2 \quad (8)$$

5.3 A Numerical Example

For receiver $x_1 = (1, 0, 0)$ and $R = I$:

- Source A $(0.8, 0.1, 0) \rightarrow \text{dist}^2 = 0.05 \rightarrow s = -0.05$
- Source B $(0, 1, 0) \rightarrow \text{dist}^2 = 2.0 \rightarrow s = -2.0$

Softmax over $[-0.05, -2.0]$ assigns $\approx 87.5\%$ weight to A and 12.5% to B.

5.4 Loss Objective and Complexity

$$\mathcal{L} = -\frac{1}{L-1} \sum_{i=1}^{L-1} \log \text{softmax}(z_i)[t_{i+1}] \quad (9)$$

The pairwise scoring requires $\mathcal{O}(L^2)$ computation in sequence length.

6 Worked Examples and Scenarios

- **Scenario A (Local Pattern):** "The sky is blue." Context drives state to predict "blue".
- **Scenario B (Order Sensitivity):** "The dog chased the cat" vs "The cat chased the dog." Position embeddings preserve sequence order.
- **Scenario C (Long Dependency):** "The keys to the cabinet... are..." Requires weight allocation across intervening tokens.
- **Scenario D (Relational Concepts):** Geometric proximity represents concept clusters (e.g., cat/dog vs car/truck).
- **Scenario E (Ambiguity):** Context transforms "bank" (river vs finance) into distinct 3D points.

7 Representational Capacity and Limitations

1. **Strict 3D Bottleneck:** Compressing syntax, entities, and context into 3 scalars risks state collisions.
2. **Deep Layer Compression:** Depth non-linearly transforms state, but state capacity remains 3D.
3. **Vocabulary Mapping:** Decoder cannot resolve contexts that collapsed to identical 3D states.

8 Complete PyTorch Reference Implementation

The complete PyTorch implementation for GLM-3 is presented below. It contains zero external dependencies beyond standard PyTorch.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class GeometricAttentionBlock(nn.Module):
    """
    Core GLM-3 Geometric Block.
    Performs affine coordinate transformation, pairwise 3D Euclidean distance calculation,
    causal softmax scaling, message aggregation, and residual state updating in R^3.
    """
    def __init__(self, dim: int = 3):
        super().__init__()
        self.dim = dim
        # Linear metric transformation R for source tokens
        self.relation = nn.Linear(dim, dim, bias=False)
        # Message transform matrix A
        self.message = nn.Linear(dim, dim, bias=False)
        # Non-linear update MLP phi: R^3 -> R^3
        self.update = nn.Sequential(
            nn.Linear(dim, dim),
            nn.Tanh(),
            nn.Linear(dim, dim)
        )
        self.norm = nn.LayerNorm(dim)
        # Softplus parameterization for learnable positive temperature tau
        self.temperature_raw = nn.Parameter(torch.tensor(1.0))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """
        Input shape:  (Batch B, Sequence Length L, 3)
        Output shape: (Batch B, Sequence Length L, 3)
        """
        B, L, D = x.shape
        assert D == self.dim, f"Input dimension must be strictly {self.dim}D, got {D}."

        # 1. Transform source states: (B, L, 3)
        source = self.relation(x)

        # 2. Compute pairwise displacement vectors: (B, L, L, 3)
        # x.unsqueeze(2) -> (B, L, 1, 3); source.unsqueeze(1) -> (B, 1, L, 3)
        delta = x.unsqueeze(2) - source.unsqueeze(1)

        # 3. Compute squared Euclidean distance scores: (B, L, L)
        scores = -delta.square().sum(dim=-1)

        # 4. Temperature scaling
        tau = F.softplus(self.temperature_raw) + 1e-4
        scores = scores / tau

        # 5. Apply causal mask (mask future tokens j > i with -inf)
        causal_mask = torch.triu(
            torch.ones(L, L, device=x.device, dtype=torch.bool), diagonal=1
        )
        scores = scores.masked_fill(causal_mask, float("-inf"))

        # 6. Normalize attention weights across visible prefix: (B, L, L)
        attn_weights = torch.softmax(scores, dim=-1)

        # 7. Aggregate messages: (B, L, 3)
```

```
        values = self.message(x)
        messages = torch.bmm(attn_weights, values)

        # 8. Non-linear state update with residual connection and LayerNorm
        updated_state = x + self.update(messages)
        return self.norm(updated_state)


class GeometricLanguageModel(nn.Module):
    """
    Full Autoregressive Geometric Language Model (GLM-3).
    Embeds discrete tokens into R^3, processes states through N Geometric Attention
        Blocks,
    and decodes final 3D coordinates into vocabulary logits.
    """
    def __init__(
        self,
        vocab_size: int,
        max_length: int = 128,
        num_layers: int = 4,
        decoder_width: int = 64
    ):
        super().__init__()
        self.vocab_size = vocab_size
        self.max_length = max_length

        # Token and position embeddings in R^3
        self.token_embedding = nn.Embedding(vocab_size, 3)
        self.position_embedding = nn.Embedding(max_length, 3)

        # Stack of strict 3D geometric attention blocks
        self.blocks = nn.ModuleList([
            GeometricAttentionBlock(dim=3) for _ in range(num_layers)
        ])

        # Vocabulary decoder MLP: R^3 -> decoder_width -> Vocab_Size
        self.decoder = nn.Sequential(
            nn.Linear(3, decoder_width),
            nn.GELU(),
            nn.Linear(decoder_width, vocab_size)
        )

    def forward(self, token_ids: torch.Tensor) -> torch.Tensor:
        """
        Input shape:  (Batch B, Sequence Length L) [LongTensor]
        Output shape: (Batch B, Sequence Length L, Vocab_Size)
        """
        B, L = token_ids.shape
        assert L <= self.max_length, f"Sequence length {L} exceeds maximum allowed length
            {self.max_length}."

        positions = torch.arange(L, device=token_ids.device)

        # Initial 3D state construction
        x = self.token_embedding(token_ids) + self.position_embedding(positions)[None, :,
            :]

        # Process through geometric blocks
        for block in self.blocks:
            x = block(x)

        # Decode 3D state to vocabulary logits
        logits = self.decoder(x)
```

```
        return logits
```

Listing 1: Complete, fully self-contained PyTorch implementation of GLM-3 (Geometric Language Model).

9 Training Objective & Protocol

Training uses standard cross-entropy over shifted targets with AdamW (lr $=$ 3e-4, weight_decay $=$ 0.01, gradient clipping $=$ 1.0). Data progression moves from tiny deterministic sequences to synthetic grammar, held-out compositional splits, and public corpora.

10 Prior Research & Novelty Positioning

Key related lines of work include Poincaré Embeddings [**?**], RotatE [**?**], Geometry-Aware Knowledge Graphs, and Tensor Field Networks [**?**]. Novelty resides in measuring the capacity of strict $\mathbb{R}^3$ states in causal autoregressive LMs rather than asserting 3D visualization capabilities.

Appendix A. Formal Algorithm

Algorithm 1: Geometric Causal Block

Input: $X \in \mathbb{R}^{B\times L\times 3}$, transforms R, A, function φ, temp τ
$Y = XR^T$
$S_{b,i,j} = -\|X_{b,i,:} - Y_{b,j,:}\|^2/\tau \quad \forall b, i, j$
Set $S_{b,i,j} = -\infty \quad \forall j > i$
$W = \text{softmax}(S, \text{axis} = -1)$
$V = XA^T$
$M = WV$
$X_{\text{new}} = \text{LayerNorm}(X + \varphi(M))$
Return X_{new}

References

[1] Maximilian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. In *Advances in Neural Information Processing Systems 30*, 2017.

[2] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. Rotate: Knowledge graph embedding by relational rotation in complex space. In *ICLR*, 2019.

[3] Nathaniel Thomas et al. Tensor field networks: Rotation- and translation-equivariant neural networks for 3d point clouds. *arXiv:1802.08219*, 2018.

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017.

[5] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv:1607.06450*, 2016.